

Date: 2017-10-08

Thomas Köppe <tkoeppe@google.com>

ISO/IEC JTC1 SC22 WG21 P0806r0

To: EWG

Deprecate Implicit Capture of `this`

Contents

1. [Revision history](#)
2. [Summary](#)
3. [Discussion](#)
4. [Proposed wording](#)
5. [Poll suggestions and early feedback](#)
6. [Future directions](#)
7. [Compatibility, upgrade path, warnings](#)

Revision history

- P0806r0: Initial proposal.

Summary

We propose to deprecate the implicit capture of `this` in a lambda expression with a *capture-default*. Users should explicitly use one of `[&, this]`, `[&, *this]`, `[=, this]` or `[=, *this]`.

Before the proposal

```
struct Foo {  
    int n = 0;  
    void f(int a) {  
        g([&, a](int k) { n  
+= a * k; });  
    }
```

With the proposal

```
struct Foo {  
    int n = 0;  
    void f(int a) {  
        g([&, this, a](int k)  
{ n += a * k; });  
    }
```

Before the proposal	With the proposal
<pre> g([=](int k) { return n + a * k; }); g([=, *this](int k) { return n + a * k; }); } }; </pre>	<pre> g([=, this](int k) { return n + a * k; }); g([=, *this](int k) { return n + a * k; }); } }; </pre>

Discussion

There are two kinds of default capture in a lambda expression:

- `[&]`: names in the lambda body refer to the actual entities that they name.
- `[=]`: names in the lambda body refer to local data members of the closure object that were copied from the entities in the surrounding scope.

When the lambda expression occurs inside a non-static class member function, any name `foo` of a non-static class member is interpreted as `this->foo`, where `this` is now bound to a local data member of the closure object. This copy is initialised from the value of `this` of the surrounding class member function *for both kinds of default capture*.

This situation is somewhat surprising and irregular. The semantics of capture-by-reference suggest that class members should refer directly to class members of the containing object. On the other hand, `this` is a pointer value, and it is clearly itself *copied* into the closure object. So whichever way one looks at the capture, one can rationalise for both `[&]` and `[=]` that they cause a copy of `this` to be stored in the closure object. By contrast, neither capture default effects the capture of `[*this]`, which would copy the entire containing object into the closure object and rebind `this` to point to that copy.

At this point, some historic background is helpful. The `this` keyword was introduced to C++ before references. At the time, C++ was translated into C, and the simple pointer semantics of `this` were convenient. If the entire feature set of references, value categories and classes had been designed together, `this` would have been an lvalue designating the object, and not a prvalue designating the address of the object. We can adopt this historic perspective by thinking of the fundamental object as “`*this`” rather than `this`. Let us consider how we might capture this object:

- Capture `*this` by value: `[*this]`
- Capture `*this` by reference: hypothetically we would say “`[&*this]`”; in C++ we say `[this]`
- If the default capture is reference capture, the hypothetical “`[&, &*this]`” (in real C++: `[&, this]`) is a redundant reference capture; `[&, *this]` is a non-redundant value capture.
- If the default capture is value capture, the hypothetical “`[=, &*this]`” (in real C++: `[=, this]`) is a non-redundant reference capture; `[&, *this]` is a redundant value capture.

In C++14, the fact that both default captures captured the `this` pointer was perhaps peculiar, but unambiguous. But C++17 added genuine value-capture of the containing object via `[*this]`, so that it is now more surprising that `[=]` means `[=, this]` and not `[=, *this]`. In other words, one default capture (`[&]`) captures `*this` in the way that would be redundant when spelled out, but the other capture (`[=]`) captures it in the non-redundant way.

This inconsistency in defaults is confusing. Users may well know that there *exists* an inconsistency, but it is much harder to know which way round the inconsistency goes. For this reason, we propose that users should never rely on implicit capture of `*this`, whether by reference or by value, and always spell out explicitly which form they want:

- `[=]` → `[=, this]`: local variables by value, class members by reference
- `[=]` → `[=, *this]`: everything by value
- `[&]` → `[&, this]`: everything by reference
- `[&]` → `[&, *this]`: (this would be unusual)

A diminished form of this proposal would be to only deprecate the implicit capture of `this` by a `[=]`-capture, since the reference capture is unambiguous and unsurprising. However, we believe that in a codebase that contains all kinds of captures, if only one version had an implicit capture and all the others did not, it would be more confusing to remember what the one exception was than if all captures would just be explicit.

Proposed wording

Append a sentence to [8.1.5.2 `expr.prim.lambda.capture`]p7:

A *lambda-expression* with an associated *capture-default* that does not explicitly capture `*this` or a variable with automatic storage duration (this excludes any *id-expression* that has been found to refer to an *init-capture*’s

associated implicit capture non-static data member), is said to *implicitly capture* the entity (i.e., `*this` or a variable) if the *compound-statement*:

- odr-uses (6.2) the entity (in the case of a variable),
- odr-uses (6.2) `this` (in the case of the object designated by `*this`), or
- names the entity in a potentially-evaluated expression (6.2) where the enclosing full-expression depends on a generic lambda parameter declared within the reaching scope of the *lambda-expression*.

The implicit capture of `*this` is deprecated; see D.?

Insert a new section [D, depr], between the current [D.3, depr.except.spec] and [D.4, depr.cpp.headers].

D.? Implicit capture of `*this` by reference [depr.capture.this]

For compatibility with prior C++ International Standards, a lambda expression with *capture-default* & or = ([8.1.5.2, expr.prim.lambda.capture]) can capture `*this` by reference implicitly.

Poll suggestions and early feedback

Feedback from the reflector on a draft of this proposal included concerns that the deprecation of `[&]` to mean `[&, this]` would break useful code in a way that has no easy upgrade, for example macros that expand to lambdas that capture everything by reference.

The following two polls would provide valuable guidance.

Poll: *Proposal as is: should we deprecate the implicit capture of `*this` by reference?*

Poll: *Should we deprecate `[=]` to mean `[=, this]` (but leave `[&]` untouched)?*

Future directions

The obvious next step would be to make the implicit capture of `*this` ill-formed. This could only happen post-C++20, since valid C++17 code must first upgrade to C++20 by changing `[=]` to `[=, this]`, which is ill-formed in C++17 (added by [P0409R2](#)). The meaning of `[=]` could then be changed again in the farther future. Such a change would presumably affect a large amount of use cases, but

the upgrade path is straight-forward. Alternatively, the feature could simply remain deprecated to guide users to avoid implicit capture of `*this` in new code.

Compatibility, upgrade path, warnings

Let us briefly examine the upgrade path and compiler warnings that would likely result from this proposed change, in the spirit of [P0684](#). The change does not break otherwise valid C++20 code, and the earliest revision in which a breakage from C++17 could appear is C++23. Therefore compilers for the current standard (C++17) need not emit *any* warning, but at best it would be a “future deprecation” warning rather than a “future breakage” warning. In C++20 conformance mode, compilers may reasonably emit a deprecation warning by default, or they could defer any warnings to the “future breakage” kind if and when a proposal for a breaking change is accepted. The replacement `[=, this]` for the deprecated `[=]` is ill-formed in C++17 and only available in C++20 as of [P0409r2](#).